



SUSTAINSYS
AI-POWERED ENGINEERING

JANUARY 2026

Getting Started: Guide to Your First Agent Using **Intelliquest Agentic Framework**

Your comprehensive guide to building intelligent AI agents — from setup to production

Contents

Agenda — Your Roadmap to Building Intelligent Agents

This comprehensive guide walks you through every step of the [Intelliquest Agentic Framework](#) — from initial setup to production deployment. Whether you're a seasoned developer or new to agentic AI, this guide provides the practical knowledge you need.

01	Objective	Understanding the purpose, scope, and outcomes of this guide
02	Getting Started	Essential skills, environment setup, and API access configuration
03	Build Your First Agent	Step-by-step agent creation, tool configuration, and task mapping
04	Deployment & Hosting	Tool hosting options, certification process, and maturity levels
05	Agent Catalogue	Publishing your agent for the wider SustainSys community
06	Support & Resources	Troubleshooting tips, FAQs, and getting expert help

Quick Reference

Est. Time	2-4 hours for complete setup	Difficulty	Intermediate
Prerequisites	Python 3.9+, API access, Postman	Support	support@sustainsys.co.uk

01 — Objective

Transform Your Ideas Into Intelligent, Autonomous Agents

The **Intelliquest Agentic Framework** represents the next evolution in AI-powered automation. This guide provides engineering teams with a comprehensive onboarding experience — enabling rapid environment setup, agent creation, and confident scaling to production.

What You Will Learn	Key Benefits
✓ Environment Setup — Configure development tools and credentials ✓ Agent Creation — Build agents with prompts, schemas, and configs ✓ Tool Integration — Connect API, script, MCP, and runtime tools ✓ Task Orchestration — Define and map tasks with reasoning ✓ Monitoring & Debugging — Track execution and troubleshoot ✓ Production Deployment — Deploy, certify, and scale agents	■ Rapid Development — Concept to agent in hours, not weeks ■ Flexible Architecture — Multiple tool types and patterns ■ Full Observability — Monitoring via State and Event APIs ■ Production Ready — Clear certification path to enterprise ■ Community Access — Publish to the Agent Catalogue ■ Expert Support — Direct access to engineering team

Real-World Insights: This guide incorporates troubleshooting steps from actual user experiences — addressing common challenges, clarifying configurations, and sharing best practices from hundreds of successful deployments.

02 — Getting Started: Essential Skills

Master the Foundational Technologies for Agentic AI Development

Building intelligent agents requires a systems-thinking approach combined with hands-on proficiency across multiple technical domains. The Intelliquest framework leverages cutting-edge AI while maintaining compatibility with industry-standard practices.

Core Programming & Automation	AI & Large Language Models	Orchestration & Workflow
Python 3.9+ FastAPI for API development Postman for API testing Swagger/OpenAPI specs JSON Schema design RESTful API patterns Async programming	OpenAI GPT models Anthropic Claude DeepSeek models RAG architectures Prompt engineering Token optimization Fine-tuning basics	LangChain framework LangGraph workflows CrewAI multi-agent State management Event-driven design Error handling patterns Retry strategies

Recommended Learning Path

Week 1-2	Python fundamentals, FastAPI basics, API testing with Postman
Week 3-4	LLM concepts, prompt engineering, RAG introduction
Week 5-6	LangChain/LangGraph, building simple chains and agents
Week 7-8	Intelliquest Framework deep-dive, first production agent

02 — Getting Started: Environment Setup

Configure Your Development Environment for the Intelliquest Framework

Prerequisites	Environment Configuration (Postman)
Required Access & Credentials: • Intelliquest Framework access (contact admin) • Environment selection: Dev, INT, or Prod • Client ID — unique identifier for your client • Project ID — your project's unique identifier • Workspace ID — your team's workspace • App Key — authentication key for API access Required Software: • Python 3.9 or higher • Postman (latest version) • Git for version control • VS Code or preferred IDE	Step-by-Step Setup: 1. Install Postman from postman.com/downloads 2. Import the Intelliquest collection (provided) 3. Create environment variables: - client_id, project_id, workspace_id - auth_token (populated after login) - app_key (provided by admin) 4. Configure pre-request scripts to auto-inject headers 5. Set base URLs for State API and Event API 6. Test connection with health check endpoint Required Headers for All Requests: • x-client-id: {{client_id}} • x-project-id: {{project_id}} • x-workspace-id: {{workspace_id}} • Authorization: Bearer {{auth_token}}

Token Generation Process

Authentication is handled via the LoginAPI endpoint. Follow these steps: 1) Call LoginAPI with app_key and redirect_url, 2) Open generated URL in browser, 3) Complete authentication, 4) Extract token from cookies, 5) Paste into Postman's auth_token variable.

Example cURL Command:

```
curl --location 'https://api.intelliquest.sustainsys.co.uk/v2/auth/login?redirect_url=...' --header 'accept: application/json' --header 'app-key: YOUR_APP_KEY'
```

03 — Build Your First Agent

Create, Configure, and Deploy Your Intelligent Agent

Agent Creation	Tools Configuration	Tasks Configuration
Use the Agent API to create: System Prompts Define the agent's persona, capabilities, and guidelines. Input Schema (JSON) Specify expected input format and validation rules. Output Schema Define response structure and format requirements. Model Configuration Select LLM provider, model version, and parameters.	Supported Tool Types: API Tool External REST APIs with authentication and retries. Script Tool Python scripts executed in isolated environments. MCP Tool Model Context Protocol tools for secure data access. MCP-Runtime Tool Dynamic MCP tools loaded at execution time.	Task Definition: Task Name Clear, descriptive identifier for the task. Description Detailed explanation of what the task accomplishes. Input/Output Specs Expected data formats for task execution. Mapping Tasks are automatically mapped to tools by the agent's reasoning engine.

Example Input Schema:

```
{"type": "object", "properties": {"repo_url": {"type": "string", "description": "Git repository URL"}, "branch": {"type": "string", "default": "main"}}, "required": ["repo_url"]}
```

Script Tool Example:

```
{"script": "import requests\nresponse = requests.get(url)\nreturn response.json()", "environment": "python", "dependencies": ["requests"]}
```

03 — Agent Execution & Monitoring

Execute Your Agent and Monitor Performance in Real-Time

Agent Execution	Monitoring & Debugging
Query Agent API Usage: • Endpoint: POST /api/v2/agents/{id}/query • Pass input matching your defined schema • Include all required headers • Handle async responses appropriately Execution Flow: 1. API validates input against schema 2. Agent processes request through reasoning 3. Tools are invoked as needed 4. Results are aggregated and returned 5. State is persisted for monitoring Response Handling: • Check status codes (200, 202, 4xx, 5xx) • Parse structured output per schema • Handle partial results gracefully • Implement retry logic for failures	Event API Tracking: • GET /api/v2/agents/{id}/events • Real-time execution visibility • Filterable by event type and time Tracked Metrics: • Tool inputs and outputs • Task execution results • Error messages and stack traces • Iteration history (for loops) • Current execution status • Token usage and latency • Memory consumption State API Features: • Persistent state across sessions • Checkpoint/restore capabilities • State inspection for debugging • Rollback to previous states

Best Practices for Production

Practice	Description	Impact
Implement Timeouts	Set appropriate timeouts for tool calls and overall execution	Prevents runaway processes
Use Structured Logging	Log all inputs, outputs, and intermediate states	Enables debugging and auditing
Handle Partial Failures	Design agents to gracefully handle tool failures	Improves reliability

Monitor Token Usage	Track LLM token consumption per request	Controls costs
Version Your Agents	Use semantic versioning for agent configurations	Enables rollback

04 — Deployment & Certification

Take Your Agent from Development to Production

Tool Hosting	Certification & Deployment
Hosting Requirements: Engineers must provision their own tool hosting. Container Deployment • Deploy API tools as Kubernetes pods • Use Docker for consistent environments • Implement health checks and scaling MCP Server Hosting • Host MCP servers for secure data access • Configure authentication/authorization • Implement rate limiting and quotas Script Tool Execution • Git-based scripts require fetch mechanism • Use dedicated tool to clone and execute • Manage dependencies via requirements.txt Infrastructure Recommendations: • Use managed Kubernetes (EKS, GKE, AKS) • Implement proper secrets management • Configure auto-scaling based on demand	Deployment Process: 1. Complete agent development and testing 2. Follow API specifications for compliance 3. Submit certification checklist 4. Complete Beta UAT with test users 5. Submit to agent board for approval 6. Deploy to production environment Certification Checklist: ■ All tests passing (unit, integration, e2e) ■ Documentation complete and reviewed ■ Security scan completed ■ Performance benchmarks met ■ Error handling verified ■ Logging and monitoring configured ■ Rollback procedure documented ■ UAT sign-off obtained Approval Timeline: • Agent board reviews weekly • Typical approval: 3-5 business days • Expedited reviews available

Maturity Levels for Agent Certification



• UX/Code prototype • Agent definition • Demo video • Stakeholder feedback	• Working MVP • Tools connected • Tasks functional • Use case validated • Team assigned	• E2E coverage • Version controlled • UAT complete • 1-3 teams using • Docs complete	• Production ready • Performance tested • Security reviewed • Broad adoption • SLA defined
--	---	--	--

04 — MCP Tools & Knowledge Resources

Understanding the Model Context Protocol for Secure AI Integration

Model Context Protocol (MCP) is an open standard that enables AI models to securely interact with external tools, data sources, and developer-defined capabilities. MCP provides a standardized way to expose functionality to LLMs while maintaining security and control.

MCP Capabilities	Integration Patterns	Security Features
Tool Invocation Agents can call external tools through MCP servers.	REST API Integration Standard HTTP/REST patterns for existing APIs.	Authentication OAuth, API keys, certificates for secure connections.
Resource Access Secure access to databases, files, and APIs.	Database Connectors Direct SQL/NoSQL access with parameterized queries.	Authorization Fine-grained permissions per tool and resource.
Context Injection Dynamic context provided to agents at runtime.	File System Access Controlled read/write to specific directories.	Audit Logging Complete audit trail of all MCP interactions.
State Management Persistent state across tool invocations.	Custom Protocols Extensible framework for proprietary systems.	Rate Limiting Prevent abuse with configurable quotas.

Essential Documentation & Learning Resources

Category	Resource	Description
Official Docs	MCP Specification	Complete protocol specification and reference
Official Docs	MCP GitHub Repository	Source code, examples, and community contributions
Integration	OpenAI Tools Guide	Building tools for ChatGPT and API integrations
Integration	Anthropic Tool Use	Claude tool use patterns and best practices

Learning	MCP Masterclass	Complete video course on MCP in Python
Community	MCP Discord	Active community for questions and discussions

05 — Publish to Agent Catalogue

Share Your Agent with the SustainSys Community

The [Intelliquest Agent Catalogue](#) is the central hub for discovering, sharing, and deploying intelligent agents across the organization. Using a familiar [YAML-first approach](#), you can easily onboard your agent and make it available to the wider community.

Publication Process	YAML Configuration	Visibility Options
Step-by-Step Publishing: 1. Complete certification 2. Create agent.yaml file 3. Add documentation 4. Submit for review 5. Publish to catalogue Required Components: • Agent metadata • Version information • Capability description • Usage examples • Access requirements • Support contact	agent.yaml Structure: <pre>name: my-agent version: 1.0.0 description: Brief description of what the agent does capabilities: - data_analysis - report_generation inputs: - name: query type: string required: true outputs: - name: result type: object</pre>	Access Levels: Public Available to all SustainSys users organization-wide. Team Limited to specific teams or departments. Private Only accessible to the publishing team. Custom Fine-grained permissions per user or group. Discovery: Searchable by capability, category, and keywords.

Documentation & Resources

Resource	Purpose	Access
Agent Onboarding Guide	Complete walkthrough for first-time publishers	Internal Wiki

Agent YAML Reference	Full specification for agent.yaml configuration	API Portal
Agents Context Store	Centralized storage for agent contexts and state	Context API
Intelliquest Agent Store	Browse and discover published agents	Catalogue Portal
Agent Homepage Template	Standard template for agent documentation pages	Template Repo

06 — Troubleshooting & Support

Resolve Issues Quickly with Expert Guidance

This section compiles common issues and solutions drawn from real user experiences. For issues not covered here, reach out to our support team who are available to help you succeed.

Issue	Symptoms	Solution
Token Expired	401 Unauthorized response	Regenerate token via LoginAPI, update Postman variable
Incorrect Headers	400 Bad Request response	Verify pre-request script, check environment variables
Tool Input Errors	Tool execution fails validation	Validate input against schema, check required fields
Execution Timeout	Agent stops responding	Increase timeout, check tool performance, add retries
State API Errors	State not persisting	Check workspace permissions, verify state key format
MCP Connection Failed	Tool cannot reach MCP server	Verify network config, check MCP server health
Rate Limited	429 Too Many Requests	Implement backoff strategy, request quota increase
Schema Mismatch	Unexpected output format	Review output schema, check agent configuration

Key Resources	Support Channels
---------------	------------------

Documentation:

- Agent Creation FAQs
- API Documentation Portal
- API Specification Reference
- Intelliquest Agent Catalogue
- Agent Homepage & Tutorials

Environments:

- Development: dev.intelliquest.sustainsys.co.uk
- Integration: int.intelliquest.sustainsys.co.uk
- Production: api.intelliquest.sustainsys.co.uk

Email Support: support@sustainsys.co.uk

Response Times:

- Critical: 2 hours
- High: 4 hours
- Medium: 1 business day
- Low: 3 business days

Office Hours: Monday-Friday, 9am-6pm GMT

Emergency support available 24/7

Community: Internal Slack [#intelliquest-support](#)

Your Feedback Matters

Help Us Improve the Intelliquest Framework Guide

We continuously improve this guide based on your feedback. Your insights help us enhance clarity, usability, and the overall developer experience. Please share your thoughts using the feedback portal.

Feedback Process	What We Track	How We Respond
How to Submit: 1. Visit the feedback portal 2. Select feedback category 3. Describe your experience 4. Add suggestions (optional) 5. Submit with contact info Categories: • Content accuracy • Clarity & readability • Missing information • Technical errors • Suggestions	Feedback Metrics: • Submission volume • Category distribution • Sentiment analysis • Recurring themes • Resolution rate Review Cycle: Weekly review by the documentation team with monthly summary reports to stakeholders.	Our Commitment: • Acknowledge within 2 days • Triage and prioritize • Update guide as needed • Notify submitter • Track in changelog Transparency: Major updates announced in #intelliquest channel.

Submit Your Feedback

feedback.sustainsys.co.uk/intelliquest

Thank you for being part of the SustainSys Consulting Ltd community. Together, we're building the future of intelligent automation.